CHƯƠNG 2

MÃ HÓA & XÁC THỰC

AES • RSA • MFA • OAUTH 2.0 • PKI • LET’S ENCRYPT



Một lần đăng nhập — nhiều lớp bảo vệ

Sinh viên truy cập cổng học tập từ Wi-Fi công cộng

01 **TLS mã hóa kênh truyền**

02 **MFA kiểm tra thêm một yếu tố**

03 **OAuth giới hạn quyền ứng dụng**

04 **Chứng chỉ gắn khóa với tên miền**

Mỗi lớp giải một câu hỏi khác nhau; thiếu một lớp có thể phá vỡ toàn bộ trải nghiệm tin cậy.

Sau chương này, sinh viên có thể…

- Phân biệt mã hóa đối xứng và bất đối xứng
- Chọn AES mode, nonce, padding và quản lý khóa phù hợp
- Dùng OpenSSL/PyCryptodome cho AES, RSA-OAEP và RSA-PSS
- Thiết kế MFA và giải thích OAuth 2.0 + PKCE
- Đọc chuỗi chứng chỉ và tự động hóa Let’s Encrypt

Hiểu → thao tác → thiết kế → đánh giá

Bản đồ học tập Chương 2

1

AES
Bảo mật dữ liệu

2

RSA
Khóa & chữ ký

3

MFA + OAuth
Danh tính & quyền

4

PKI + ACME
Niềm tin trên web

Khởi động: chọn phát biểu đúng nhất

Mã hóa và xác thực khác nhau ở đâu?

A. Mã hóa chứng minh danh tính

B. Xác thực luôn giữ dữ liệu bí mật

C. Mã hóa bảo vệ nội dung; xác thực kiểm tra chủ thể

D. Hai khái niệm hoàn toàn giống nhau

Đáp án C — trong hệ thống thực tế, chúng phối hợp nhưng không thay thế nhau.

PHẦN 1

AES & RSA

Từ khối dữ liệu, khóa phiên đến chữ ký số



Mã hóa, băm và chữ ký số không đồng nghĩa

GÓC NHÌN 1

- Mã hóa: có thể giải mã bằng khóa
- Băm: một chiều, tạo digest
- MAC: toàn vẹn + khóa bí mật

GÓC NHÌN 2

- Chữ ký số: khóa riêng ký
- Khóa công khai xác minh
- Không tự động giữ nội dung bí mật

Đối xứng và bất đối xứng bổ trợ nhau

GÓC NHÌN 1

- Một khóa bí mật chung
- Nhanh, phù hợp dữ liệu lớn
- Bài toán: phân phối khóa

GÓC NHÌN 2

- Cặp khóa công khai/riêng
- Chậm hơn, phù hợp khóa nhỏ/chữ ký
- Bài toán: tin khóa công khai

AES là mã khối đối xứng tiêu chuẩn

Khối 128 bit • khóa 128 / 192 / 256 bit

AES biến đổi từng khối 128 bit qua nhiều vòng. AES-128, AES-192 và AES-256 lần lượt dùng 10, 12 và 14 vòng.

SubBytes

ShiftRows

MixColumns

Ba kích thước khóa — cùng kích thước khối

AES-128

10 vòng

Hiệu năng tốt; dùng rộng rãi

AES-192

12 vòng

Ít phổ biến hơn

AES-256

14 vòng

Biên an toàn khóa lớn hơn

Mode quyết định cách AES xử lý nhiều khối

Mode	Bí mật	Toàn vẹn	Lưu ý
ECB	Có hạn chế	Không	Lộ mẫu
CBC	Có	Không	IV + padding
CTR	Có	Không	Nonce không lặp
GCM	Có	Có	AEAD; ưu tiên thực hành

ECB làm lộ cấu trúc dữ liệu

Khối giống nhau → ciphertext giống nhau

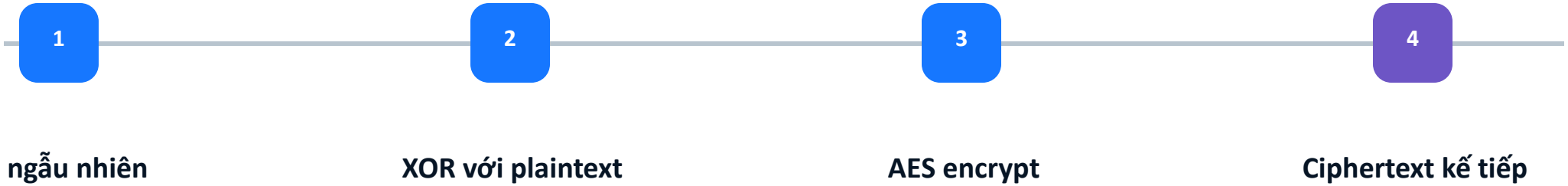
ECB không trộn thông tin giữa các khối. Vì vậy hình ảnh, biểu mẫu hoặc bản ghi lặp có thể để lộ mẫu dù nội dung đã bị mã hóa.

Deterministic

Pattern leak

Avoid

CBC liên kết mỗi khối với khối trước



CBC cần padding và cơ chế xác thực riêng; sai thông báo lỗi có thể tạo padding oracle.

CTR biến AES thành dòng khóa



Nonce + counter

AES tạo keystream

XOR plaintext

Ciphertext

Không cần padding; tuyệt đối không lặp nonce/counter với cùng khóa.

GCM cung cấp mã hóa có xác thực

Confidentiality + Integrity + Authenticity

AES-GCM trả về ciphertext và authentication tag; có thể xác thực thêm AAD mà không mã hóa AAD.

Nonce

Ciphertext

Tag

Nonce, IV và salt có vai trò khác nhau

GÓC NHÌN 1

- Nonce/IV: làm mỗi lần mã hóa khác nhau
- Không nhất thiết bí mật
- Quy tắc ngẫu nhiên/duy nhất tùy mode

GÓC NHÌN 2

- Salt: đầu vào KDF cho mật khẩu
- Chống bảng băm dựng sẵn
- Lưu cùng dữ liệu dẫn xuất được

Nonce reuse trong GCM có thể phá bảo mật

Một khóa + một nonce: chỉ dùng một lần

Lặp nonce làm lộ quan hệ giữa plaintext và có thể phá khả năng xác thực. Thiết kế bộ đếm, random đủ dài và cơ chế chống reset.

Unique

Persist

Monitor

Padding chỉ cần cho một số mode

GÓC NHÌN 1

- CBC cần dữ liệu đủ khối
- PKCS#7 thêm N byte giá trị N
- Luôn kiểm tra lỗi đồng nhất

GÓC NHÌN 2

- CTR/GCM hoạt động như dòng
- Không cần PKCS#7
- Tag thất bại → từ chối toàn bộ

Khóa quan trọng hơn việc chọn tên thuật toán

01	**Sinh khóa**	CSPRNG; độ dài phù hợp
02	**Lưu khóa**	KMS/HSM; tách khỏi dữ liệu
03	**Sử dụng**	Quyền tối thiểu; audit
04	**Luân chuyển**	Phiên bản hóa; thu hồi; hủy an toàn

Không hard-code khóa trong mã nguồn hoặc đẩy khóa lên Git.

OpenSSL: mã hóa AES từ mật khẩu

```
openssl enc -aes-256-cbc -salt -pbkdf2 \
  -in diem.csv -out diem.csv.enc

openssl enc -d -aes-256-cbc -pbkdf2 \
  -in diem.csv.enc -out diem.csv
```

`enc` phù hợp minh họa CBC; OpenSSL không hỗ trợ GCM qua lệnh enc. Ứng dụng nên dùng API AEAD.

OpenSSL: kiểm tra sai mật khẩu và tính toàn vẹn

```
sha256sum diem.csv
openssl enc -d -aes-256-cbc -pbkdf2 \
  -in diem.csv.enc -out thu.csv
sha256sum thu.csv
```

CBC encryption không tự xác thực dữ liệu. Hash đối chiếu chỉ hữu ích nếu digest gốc được bảo vệ.

PyCryptodome: AES-GCM tối thiểu

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

key = get_random_bytes(32)
c = AES.new(key, AES.MODE_GCM)
ct, tag = c.encrypt_and_digest(data)
# Lưu: c.nonce + tag + ct
```

Khi giải mã: tạo cipher với nonce, rồi decrypt_and_verify(ct, tag). Nếu tag sai, không sử dụng plaintext.

Thực hành AES: nhìn thấy điều gì khi dữ liệu đổi?

Hạng mục	Nhiệm vụ
Bước 1	Mã hóa cùng file hai lần; so sánh ciphertext
Bước 2	Sửa 1 byte ciphertext; thử giải mã GCM
Bước 3	Ghi lại nonce, tag, ciphertext
Bước 4	Giải thích vì sao tag thất bại

Sản phẩm nộp: lệnh/mã nguồn + ảnh kết quả + 5 câu giải thích.

Checkpoint AES

Yêu cầu nào quan trọng nhất khi dùng AES-GCM?

A. Giữ nonce bí mật

B. Không lặp nonce với cùng khóa

C. Thêm PKCS#7 padding

D. Dùng ECB trước GCM

Đáp án B — nonce thường được gửi công khai nhưng phải duy nhất theo khóa.

RSA dùng một cặp khóa có quan hệ toán học

Public key chia sẻ • private key giữ bí mật

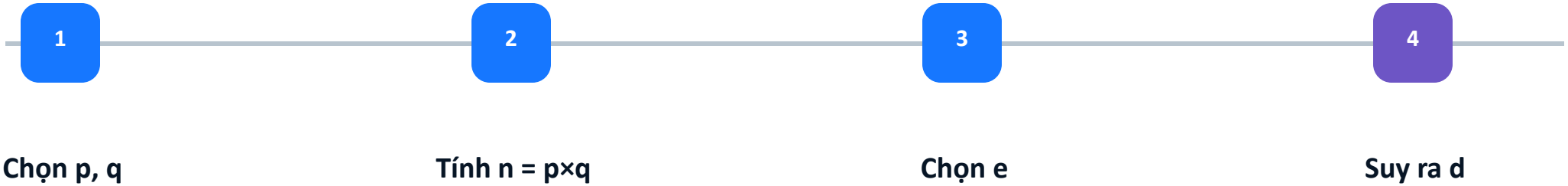
Khóa công khai có thể dùng để mã hóa hoặc xác minh; khóa riêng dùng để giải mã hoặc ký — tùy scheme.

n

e

d

Trực giác RSA: dễ nhân, khó phân tích thừa số



Thực tế phải dùng thư viện chuẩn; không tự cài đặt số học RSA cho hệ thống thật.

Sinh khóa RSA là một quy trình an toàn

01	**Entropy**	Nguồn ngẫu nhiên mật mã
02	**Kích thước**	Tối thiểu theo chính sách; thường 2048+ bit
03	**Định dạng**	PKCS#8 cho private key
04	**Bảo vệ**	Mật khẩu mạnh, quyền file, HSM/KMS

Khóa riêng bị lộ đồng nghĩa mất khả năng bí mật hoặc giả mạo chữ ký.

RSA không mã hóa trực tiếp dữ liệu lớn

Giới hạn kích thước + chi phí cao

RSA thường bảo vệ khóa phiên AES. Dữ liệu thực được AES-GCM mã hóa — đây là hybrid encryption.

Small key

AES payload

Envelope

RSA-OAEP là scheme mã hóa có padding

OAEP ngẫu nhiên hóa ciphertext

OAEP dùng hàm băm và mask generation function; tránh dùng raw RSA. OAEP không cung cấp chữ ký.

Encrypt

Randomized

Not signature

Mã hóa và ký dùng hướng khóa khác nhau

GÓC NHÌN 1

- Mã hóa cho B: dùng public key của B
- Chỉ B có private key giải mã
- Mục tiêu: bí mật

GÓC NHÌN 2

- A ký: dùng private key của A
- Mọi người dùng public key A xác minh
- Mục tiêu: nguồn gốc + toàn vẹn

RSA-PSS là scheme chữ ký hiện đại

Hash → PSS encode → RSA signature

Người xác minh tính lại digest và kiểm chữ ký bằng public key. PSS dùng salt để tăng an toàn chứng minh.

SHA-256

Salt

Verify

OpenSSL: sinh khóa và RSA-OAEP

```
openssl genpkey -algorithm RSA \
  -pkeyopt rsa_keygen_bits:3072 -out private.pem
openssl pkey -in private.pem -pubout -out public.pem
openssl pkeyutl -encrypt -pubin -inkey public.pem \
  -pkeyopt rsa_padding_mode:oaep -in key.bin -out key.enc
```

Giải mã dùng -decrypt với private.pem. Chỉ mã hóa dữ liệu nhỏ như khóa phiên.

OpenSSL: ký và xác minh SHA-256

```
openssl dgst -sha256 -sign private.pem \
  -out message.sig message.txt
openssl dgst -sha256 -verify public.pem \
  -signature message.sig message.txt
```

Trong ứng dụng mới, cấu hình RSA-PSS rõ ràng hoặc dùng API EVP phù hợp; không nhầm ký với “mã hóa bằng khóa riêng”.

PyCryptodome: OAEP và PSS

```
cipher = PKCS1_OAEP.new(pub, hashAlgo=SHA256)
wrapped = cipher.encrypt(aes_key)

h = SHA256.new(message)
sig = pss.new(private).sign(h)
pss.new(public).verify(h, sig)
```

Bắt ngoại lệ ValueError khi OAEP/PSS kiểm tra thất bại; không tiếp tục xử lý dữ liệu không xác thực.

Hybrid encryption kết hợp tốc độ và phân phối khóa

1

Sinh khóa AES phiên

2

AES-GCM mã hóa dữ liệu

3

RSA-OAEP bọc khóa AES

4

Gửi envelope

Envelope gồm wrapped key + nonce + tag + ciphertext + metadata thuật toán.

Chọn AES hay RSA theo nhiệm vụ

Nhiệm vụ	AES	RSA	Khuyến nghị
Dữ liệu lớn	Tốt	Không	AES-GCM
Chia sẻ khóa phiên	Cần kênh	Tốt	RSA-OAEP
Chữ ký	Không	Tốt	RSA-PSS
Hiệu năng	Nhanh	Chậm	Kết hợp

Các lỗi mật mã thường gặp

GÓC NHÌN 1

- Tự thiết kế thuật toán/protocol
- ECB hoặc CBC không MAC
- Nonce reuse
- Hard-code key

GÓC NHÌN 2

- Raw RSA / PKCS#1 v1.5 encryption
- Không kiểm tra tag/signature
- Thông báo lỗi quá chi tiết
- Không có kế hoạch rotation

Mini case: gửi bảng điểm cho đối tác

File 80 MB cần giữ bí mật, phát hiện sửa đổi và chỉ đối tác được mở.

1. Thuật toán nào xử lý file?
2. Khóa phiên được chuyển thế nào?
3. Metadata nào phải gửi kèm?

Đề xuất: AES-GCM cho file; RSA-OAEP bọc khóa; ký envelope nếu cần xác minh người gửi.

Checkpoint mật mã

Cách tốt nhất để bảo vệ file 500 MB cho người nhận có RSA public key?

A. RSA mã hóa toàn file

B. AES-GCM file; RSA-OAEP mã hóa khóa AES

C. Hash file rồi bỏ plaintext

D. ECB rồi ký nonce

Đáp án B — mô hình mã hóa lai.

PHẦN 2

MFA & OAuth 2.0

Xác thực chủ thể — ủy quyền ứng dụng



Ba nhóm yếu tố xác thực

Biết

Mật khẩu / PIN

Something you know

Có

Điện thoại / khóa bảo mật

Something you have

Là

Vân tay / khuôn mặt

Something you are

2FA là một trường hợp của MFA

Hai yếu tố phải thuộc hai loại khác nhau

Hai mật khẩu không phải 2FA. Một thiết bị mật mã được mở bằng sinh trắc học có thể là multi-factor authenticator.

Distinct factors

Independent

Bound

Mật khẩu đơn lẻ thất bại theo nhiều đường

Phishing • reuse • credential stuffing • malware

MFA làm kẻ tấn công phải vượt qua thêm yếu tố, nhưng sức mạnh phụ thuộc phương thức và quy trình khôi phục.

Prevent

Detect

Recover

Không phải phương thức MFA nào cũng mạnh như nhau

Phương thức	Tiện dụng	Chống phishing	Rủi ro
SMS OTP	Cao	Thấp	SIM swap
TOTP app	Khá	Thấp	Phishing relay
Push	Cao	Trung bình	MFA fatigue
FIDO/passkey	Cao	Cao	Quản lý thiết bị

Phishing-resistant MFA ràng buộc với dịch vụ thật

FIDO2 / WebAuthn / passkeys

Authenticator ký thử thách cho đúng origin; bí mật không được gửi cho website giả. NIST yêu cầu cung cấp lựa chọn chống phishing ở AAL2.

Origin bound

Challenge

Public key

Xác thực thích ứng dùng ngữ cảnh để tăng/giảm ma sát



Tín hiệu: thiết bị, vị trí, IP, thời gian, hành vi, độ nhạy tài nguyên.

Vòng đời MFA phải bao gồm khôi phục

01	**Đăng ký**	Xác minh người dùng; 2 thiết bị nếu phù hợp
02	**Sử dụng**	Rate limit; chống replay
03	**Mất thiết bị**	Recovery code; helpdesk xác minh
04	**Thu hồi**	Hủy binding; thông báo người dùng

Kẻ tấn công thường nhắm quy trình reset yếu thay vì phá thuật toán.

Push bombing biến sự tiện lợi thành áp lực

Nhiều yêu cầu → người dùng bấm “Approve”

Giảm thiểu bằng number matching, rate limit, hiển thị ngữ cảnh, cảnh báo và ưu tiên phương thức chống phishing.

Context

Rate limit

Report

Bài tập: thiết kế MFA cho cổng sinh viên

Sinh viên có điện thoại; giảng viên xử lý điểm; quản trị viên truy cập hệ thống lõi.

1. Nhóm nào cần assurance cao nhất?
2. Phương thức chính và dự phòng?
3. Khôi phục tài khoản thế nào?

Không dùng một chính sách cho mọi vai trò; cân bằng rủi ro, chi phí và khả năng tiếp cận.

OAuth 2.0 là khung ủy quyền

Delegated access — không phải giao thức xác thực

Ứng dụng nhận quyền truy cập giới hạn tới tài nguyên mà không cần biết mật khẩu người dùng. Đăng nhập thường cần OpenID Connect bên trên OAuth.

Authorization

Scoped

Token

Bốn vai trò cốt lõi của OAuth 2.0

Resource Owner

Người sở hữu quyền

Client

Ứng dụng xin quyền

Authorization Server

Cấp code/token

Resource Server

API bảo vệ tài nguyên

Authorization Code Flow tách trình duyệt và token



Client chuyển hướng

Người dùng đồng ý

Nhận authorization code

Đổi code lấy token

Code ngắn hạn đi qua front channel; token nhận tại token endpoint.

PKCE ràng buộc authorization code với client

code_verifier → SHA-256 → code_challenge

Client gửi challenge trước; khi đổi code phải chứng minh verifier. Kẻ đánh cắp code không có verifier sẽ thất bại.

S256

Verifier

Binding

Access token và refresh token có vòng đời khác nhau

GÓC NHÌN 1

- Access token: gọi API
- Ngắn hạn
- Giới hạn audience/scope
- Có thể là bearer token

GÓC NHÌN 2

- Refresh token: xin access token mới
- Dài hơn, nhạy cảm hơn
- Rotation / sender-constrained
- Thu hồi khi rủi ro

Scope thể hiện quyền tối thiểu có chủ đích

read:profile ≠ write:grades

Consent phải rõ ứng dụng nào, dữ liệu nào, mục đích nào. Resource server vẫn phải kiểm tra audience, scope và quyền nghiệp vụ.

Least scope

Audience

Policy

OAuth bị tấn công ở redirect, code và token

GÓC NHÌN 1

- Redirect URI lỏng
- CSRF / mix-up
- Code interception
- Open redirect

GÓC NHÌN 2

- Token leakage trong URL/log
- Bearer token bị replay
- Refresh token không rotation
- Scope quá rộng

Checklist OAuth 2.0 hiện đại

01	**Flow**	Authorization Code + PKCE
02	**Redirect**	So khớp URI chính xác; state
03	**Token**	TLS; ngắn hạn; rotation
04	**API**	Kiểm audience, issuer, scope; không tin client

RFC 9700 loại bỏ hoặc hạn chế nhiều mô hình cũ như implicit grant trong thiết kế mới.

Thực hành OAuth: đọc một yêu cầu ủy quyền

Hạng mục	Nhiệm vụ
client_id	Ứng dụng nào đang xin quyền?
redirect_uri	Kết quả được trả về đâu?
scope	Ứng dụng xin quyền gì?
state + PKCE	Chống CSRF và chặn code interception

Nhóm khoanh vùng tham số có thể bị sửa và nêu kiểm tra phía authorization server.

Checkpoint MFA & OAuth

Phát biểu nào đúng?

A. OAuth tự xác thực người dùng

B. SMS OTP chống phishing tuyệt đối

C. PKCE ràng buộc code với client

D. Scope càng rộng càng an toàn

Đáp án C. OAuth là ủy quyền; OIDC mới bổ sung lớp danh tính đăng nhập.

PHẦN 3

PKI & CHỨNG CHỈ SỐ

Chuỗi tin cậy, TLS và tự động hóa Let’s Encrypt



PKI gắn danh tính với khóa công khai

CA

Ký và phát hành chứng chỉ

RA

Kiểm tra yêu cầu/danh tính

Repository

Phân phối cert/CRL

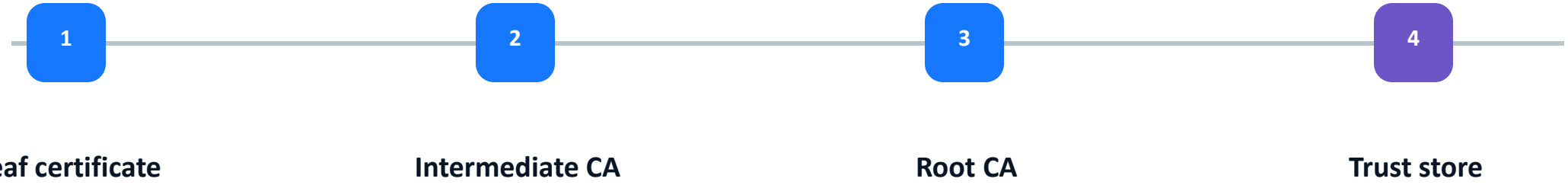
Relying party

Xác minh và quyết định tin

Một chứng chỉ X.509 chứa gì?

Trường	Ý nghĩa	Ví dụ	Câu hỏi
Subject / SAN	Danh tính	example.edu	Đúng tên miền?
Issuer	CA phát hành	Intermediate CA	Có tin CA?
Validity	Thời gian	NotBefore/After	Còn hạn?
Public key	Khóa + thuật toán	RSA/ECDSA	Đủ mạnh?
Extensions	Ràng buộc dùng	Key Usage	Đúng mục đích?

Chuỗi tin cậy đi từ leaf tới trust anchor



Máy chủ gửi leaf và intermediate; root thường đã có sẵn trong hệ điều hành/trình duyệt.

Trình xác minh kiểm nhiều điều kiện

01	**Tên**	SAN khớp hostname
02	**Chữ ký**	Mỗi cert được issuer ký hợp lệ
03	**Thời gian**	Nằm trong validity
04	**Chính sách**	Key usage, basic constraints, revocation

Chỉ “có chứng chỉ” không đồng nghĩa chứng chỉ hợp lệ cho kết nối hiện tại.

TLS 1.3 dùng chứng chỉ để xác thực máy chủ



ClientHello

ServerHello + Certificate

CertificateVerify

Finished → khóa phiên

Chứng chỉ không trực tiếp mã hóa toàn bộ phiên; handshake thiết lập khóa đối xứng cho dữ liệu ứng dụng.

Let’s Encrypt + ACME tự động hóa cấp chứng chỉ



Let’s Encrypt là CA miễn phí, tự động và mở; ACME client thực hiện phần lớn quy trình.

Chọn challenge theo kiến trúc

Challenge	Cách chứng minh	Phù hợp	Lưu ý
HTTP-01	File qua cổng 80	Web đơn giản	Không wildcard
DNS-01	TXT record DNS	Wildcard / nhiều máy	Bảo vệ API DNS
TLS-ALPN-01	TLS đặc biệt cổng 443	Hệ thống tích hợp	Ít client hỗ trợ

Chứng chỉ ngắn hạn đòi hỏi tự động hóa

Mặc định hiện tại: 90 ngày

Let’s Encrypt khuyến nghị gia hạn chứng chỉ 90 ngày sau khoảng 60 ngày. Lộ trình ngành sẽ giảm thời hạn tối đa; đến 2028 Let’s Encrypt dự kiến 45 ngày.

Auto renew

Monitor

Reload

Tình huống tích hợp: triển khai portal.example.edu

Cổng mới cần HTTPS, đăng nhập trường và cho ứng dụng lịch đọc thời khóa biểu.

1. AES/RSA xuất hiện ở lớp nào?
2. MFA áp dụng cho ai?
3. OAuth scope nào tối thiểu?
4. ACME challenge nào phù hợp?

Thiết kế phải bao gồm rotation token, renew certificate, bảo vệ private key và giám sát lỗi.

Kết thúc chương: ghép đúng lớp, đúng mục tiêu

01 AES-GCM bảo vệ dữ liệu lớn; nonce không lặp

02 RSA-OAEP bọc khóa; RSA-PSS ký

03 MFA mạnh ưu tiên khả năng chống phishing

04 OAuth 2.0 ủy quyền với Code + PKCE + scope hẹp

05 PKI và ACME tạo niềm tin có thể tự động vận hành

Exit ticket: vẽ kiến trúc cổng sinh viên và đánh dấu 01 khóa, 01 token, 01 chứng chỉ, 01 điểm xác thực.